

COMP25212 Exercise 2 - Branch Prediction Answers

Ahmad Syahrul Azim Bin Ahmad Azmi

March 2025

Phase 1 : Profiling and Statistics

1. Firstly, the instr_profile is sorted using sort command and stored in new text file (sorted_prof). Then, the number of instructions is calculated using wc. Number of instruction with cumulative 90% of accesses is calculated from sorted_prof using Java program (Branch.java).

Answer : $(1941 / 11339) * 100 = 0.1712$ or 17%

2. **Answer :** 428065

3. **Answer :** $3423933 / 428065 = 7.999 \approx 8$

4. Total number of cycles (without penalty) is 3423933. Total penalty is number of taken branch $\times 2 = 856130$. Total time = total penalty + total without penalty = $856130 + 3423933 = 4280063$. Proportion of time wasted = total penalty / total time.

Answer : 0.20 or 20%

5. **Answer :** $596271 / 667867 = 0.8928$ or 89%

6. **Answer :** 0 - when the branches are predicted perfectly, the required instruction will already be fetched from memory for execution when needed, hence no wasted cycle (or penalty) for fetching the wrong instruction.

7. **Answer :** Branch predictors might have really small window of past branch outcomes, that might not fully describe the behaviour of the branch.

8. **Answer :** 1.8

9. **Answer :** 1.1

10. **Answer :** Forward - 0.78, backward - 2.5

11. **Answer :** Assume that branch is taken for backward branch, assume branch is not taken for forward branch.

Phase 2 : Modelling

12. **Answer :** Highly associative - small caches require high associativity to reduce conflicts, and avoid underutilisation.

13. **Answer :** 1 - each branch address only has one target, and the processor can only fetch one instruction at time.

14. Policy explanation :

- Cache all types (conditional or unconditional branches)
- Only cache backward branch
- Only cache taken branch on first encounter
- Cyclic replacement is used to evict cache content

Code :

```
public class Model {
    ArrayList<Long> branch_address;
    ArrayList<Long> target_address;
    ArrayList<Integer> hysteresis;
    int insert_pointer;

    Model(int size) {
        this.branch_address = new ArrayList<Long>();
        this.target_address = new ArrayList<Long>();
        this.hysteresis = new ArrayList<Integer>();
        this.insert_pointer = 0;

        for (int i = 0; i < size; i++) {
            this.branch_address.add(0L);
            this.target_address.add(0L);
        }
    }

    private void increment_pointer() {
        this.insert_pointer += 1;
        if (this.insert_pointer == this.branch_address.size()) {
            this.insert_pointer = 0;
        }
    }

    private void add_target_address(Long branch_address_in,
                                    Long target_address_in,
                                    Boolean branch_decision) {
        if (branch_decision & (target_branch < branch_address_in)) {
            this.branch_address.set(insert_pointer, branch_address_in);
            this.target_address.set(insert_pointer, target_address_in);
            this.increment_pointer();
        }
    }

    private Boolean branch_decider(Long branch_to_search) {
        for (int i = 0; i < this.branch_address.size(); i++) {
            if (this.branch_address.get(i).equals(branch_to_search)) {
                return true;
            }
        }
    }
}
```

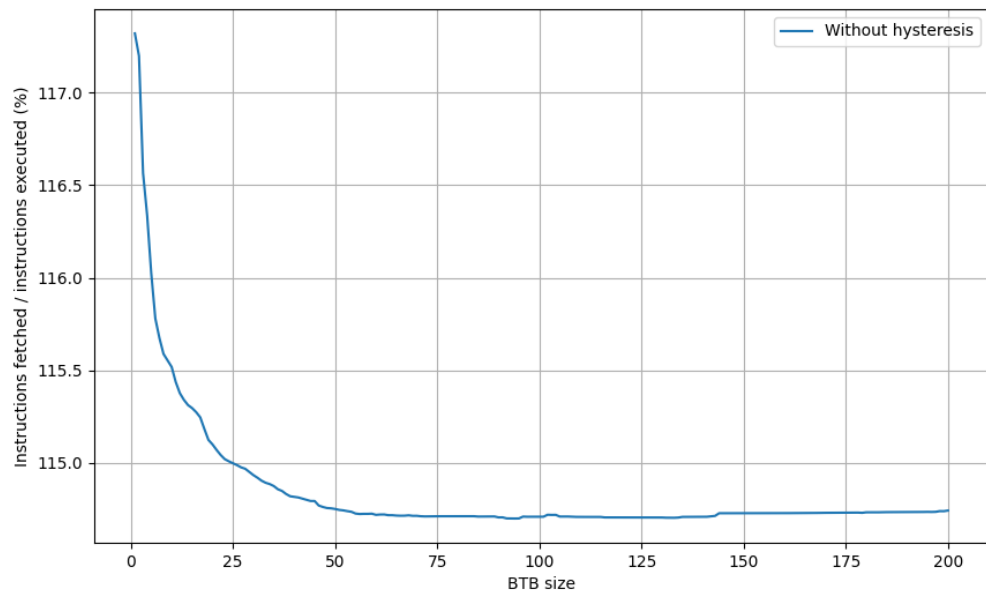
```

    }
    return false;
}

public Boolean run_branch(Long branch_to_search) {
    Boolean branch_taken = this.branch_decider(branch_to_search);
    if (branch_taken) {
        return true;
    } else {
        this.add_target_address(branch_to_search,
                                target_address, branch_condition);
        return false;
    }
}
}
}

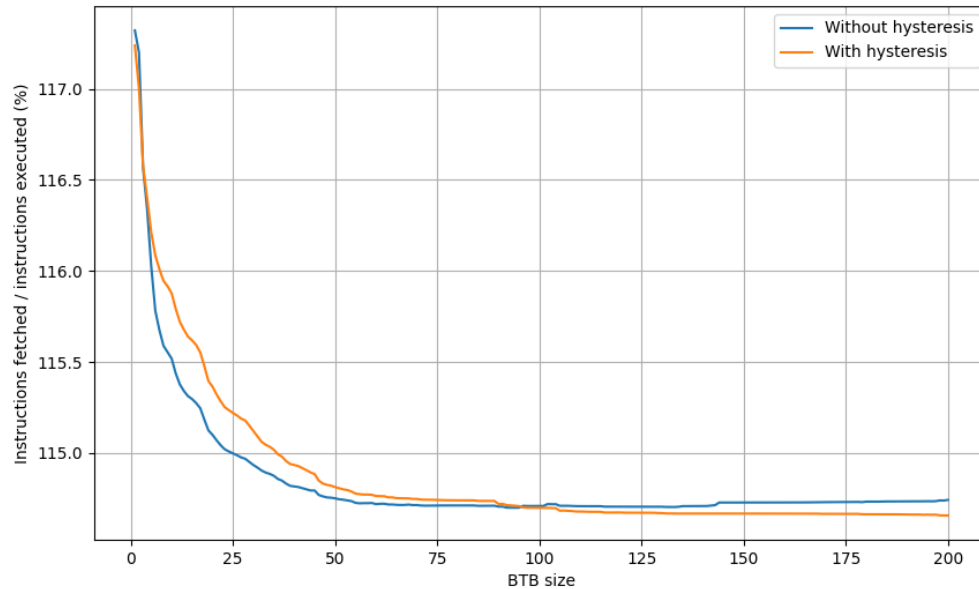
```

15. Graph :



Phase 3 : Sophistication

16. Graph :



17. **Answer :** 0.59049 or 59%

18. **Answer :**

Prediction accuracy has been mainly improved by research on different implementation methods. In addition to one-level predictor that we have implemented in the exercise, two-level predictor (or also called as Correlation-Based Branch Predictor) has also been used. It uses a two-dimensional table of counters, also called Pattern History Table. It keeps a longer history of branch history in a shift register, where the branch history (first level) is used as index to access pattern history table (second level). Some implementation using a neural network at the second level has also been proposed. With branch history, the prediction takes into account the certain context of whether a branch is taken or not. This implementation can further be improved by using saturating counts, providing hysteresis.

19. **Answer :**

"R" branches are usually return instruction from a function (as it is related to Link Register). This type of branch is easy to predict as the value of target address is already stored in the Link Register, and return call is certain. Processors might have Return Address Stack (RAS) to keep the addresses of the caller of method calls (by pushing the address on the stack), which can be popped of when return instruction is encountered. "M" branches are also related to function returns, where stack is popped into registers (such as POP {R1-R3, PC}, R1 to R3 could be the caller saved parameters, while PC is pushed to save return address). This type of branch can also be predicted using RAS. "D" branches are indirect branch, where the argument specifies where the address is located rather than specifying the address directly. Current processors use a branch target buffer (BTB) to predict indirect branches, where the branch address is used as the key to access the target address of the branch (same implementation as invariant branch), taken from history of the branch. "L" branches means that the target address is loaded from memory, hence the target is unknown until the memory operation completes. In hardware, prediction could again be implemented using a BTB, where target address is predicted based on the history of the branch. In addition to RAS and

BTB mentioned, global history-based predictors can be used to dynamically predict these branches, by considering recently executed branches to predict whether a particular branch is taken (could be a one-level or two-level predictor).

20. **Answer :** Switch-case statement - the instruction (ADDLS R15, R15, R0 LSL #2) calculate its target address based on value inside a jump table.